CS300 – Sabancı University Homework 2

PLEASE NOTE:

- The use of any form of AI is strictly prohibited.
- All solutions must be entirely your own work.
- No collaboration or cooperation among students is permitted.
- Submissions must be made through the SUcourse. No other method of submission will be accepted.

1. Introduction

A **point quadtree** is a multidimensional generalization of a binary search tree that is designed to store and query points in two-dimensional space. Such structures are especially useful when the data has a spatial interpretation and we want to support location-based operations more efficiently than by scanning all points one by one.

In this assignment, you will implement a dynamic point quadtree for a **hypothetical ride-hailing system**. Each driver will be represented as a point in a 2D service area, and the initial set of drivers will be inserted into the tree in the exact order they are given. Since the shape of a point quadtree depends on insertion order, this order is important.

After constructing the initial tree, your program will process a sequence of operations that may modify or query the structure. These operations include adding a new driver, removing an existing driver, printing the current tree, finding all drivers within a circular request region, and finding the nearest driver to a given location. For query operations, your program must also report the drivers visited during the search.

The purpose of this assignment is to give you practice with recursive tree structures, spatial partitioning, dynamic updates, and geometric searching in two dimensions. You are expected to use the point quadtree structure as the basis of your solution.

2. Program Specifications

Your program should read two text files, *drivers.txt* and *operations.txt*. The former specifies a 2D service area and lists the drivers initially located within it, while the latter specifies the operations to be performed on the dynamic point quadtree.

Input Format

File: drivers.txt :

The first line in *drivers.txt* specifies the 2D service area by giving the coordinates of the upper-right corner of the space in the form:

<x coordinate> <y coordinate>

The origin (0, 0) corresponds to the lower-left corner of the service area, and negative coordinate values are not valid. Each subsequent line in drivers.txt represents a driver, formatted as follows:

<driver id> <x coordinate of the driver> <y coordinate of the driver>

Driver IDs contain no spaces, and coordinates cannot be negative. You may assume that all driver coordinates lie within the specified 2D space.

File: operations.txt :

Each line in *operations.txt* corresponds to a single operation. The possible operation formats are:

PRINT
ADD <driver id> <x coordinate> <y coordinate>
REMOVE <driver id>
REQUEST <x coordinate> <y coordinate> <radius>
NEAREST <x coordinate> <y coordinate>

The REQUEST operation represents a circular search region centered at the given point, with the specified radius. The NEAREST operation asks for the driver whose Euclidean distance to the given point is minimum.

Point Quadtree Construction

Given the *drivers.txt* file, your program should insert the drivers into an initially empty point quadtree in the exact order they appear in the file. Since the structure of a point quadtree depends on insertion order, preserving this order is essential.

When inserting a driver, if its x-coordinate is equal to the x-coordinate of the current node, it should be considered to belong to the eastern subspace. Similarly, if its y-coordinate is equal to the y-coordinate of the current node, it should be considered to belong to the northern subspace.

Tree Printing

After constructing the tree, your program should pretty print the quadtree using the following recursive algorithm:

```
pretty_print(root):   // Pretty print the quadtree rooted at root
if root != NULL:      // If the tree is not empty
   print the driver id stored at the root
   pretty_print(root.SE)   // Recursively print the southeast subtree
   pretty_print(root.SW)   // Recursively print the southwest subtree
   pretty_print(root.NE)   // Recursively print the northeast subtree
   pretty_print(root.NW)   // Recursively print the northwest subtree
```

For a PRINT operation, your program should first print:

Tree:

Then the driver IDs should be printed according to the recursive traversal above, one per line. If the tree is empty, your program must print: <Empty>

2.1 Add Operation

The ADD operation inserts a new driver with the given ID and coordinates into the current point quadtree. If the given driver ID already exists in the tree, ignore the operation and leave the tree unchanged.

The ADD operation does not produce any output.

2.2 Remove Operation

The REMOVE operation removes the driver with the given ID from the current point quadtree. To remove a driver, first locate the node with the given ID. If no such driver exists, ignore the operation.

If the driver exists, consider the subtree rooted at that node. Remove the root of that subtree, collect all remaining nodes in that subtree (excluding the removed node) using preorder traversal in the order current node, then SE, SW, NE, NW, and reinsert the collected nodes to rebuild a valid point quadtree for that subtree.

This rule applies whether the removed node is the root of the whole tree or a non-root node. If the removed node is the root, the entire tree is rebuilt from the remaining nodes of its subtree. If the removed node is not the root, only the corresponding child subtree is rebuilt.

The REMOVE operation does not produce any output.

2.3 Request Operation

The REQUEST operation performs a circular range search on the current point quadtree. For each REQUEST, your program should produce two lines of output.

First Output Line
The first line should report the drivers whose Euclidean distance to the query point is less than or equal to the given radius.

If one or more drivers are within the specified radius, print their IDs as a comma-separated list in the order they are found during the recursive search, in the following format:

InRange: <comma-separated driver IDs>

If no driver lies within the specified radius, your program must print:

InRange: <None>

Second Output Line
The second line should report the drivers visited during the search, as a comma-separated list, in the exact order they are visited by the recursive search procedure, in the following format:

Visited: <comma-separated driver IDs>

If no node is visited, print:

Visited:

For each REQUEST, perform a recursive range search starting from the root. Whenever a node is reached by the recursive search, it is considered visited immediately and its driver ID must be appended to the Visited list. Then check whether that driver lies within the circular query region and, if so, also report it in the *InRange* list. If multiple child subtrees must be searched, they must be visited in the order SE, SW, NE, NW. A child subtree should be searched only if its subspace intersects the circular query region.

2.4 Nearest Operation

The NEAREST operation finds the driver whose Euclidean distance to the given query point is minimum. For each NEAREST, your program should produce two lines of output.

First Output Line
The first line should report the nearest driver in the following format:

Nearest: <driver id>

If the tree is empty, your program must print:

Nearest: <None>

Second Output Line
The second line should report the drivers visited during the search, as a comma-separated

list, in the exact order they are visited by the recursive search procedure, in the following format:

Visited: <comma-separated driver IDs>

If no node is visited, print:

Visited:

If multiple drivers have the same minimum Euclidean distance to the query point, report the one encountered first during the recursive traversal. For each NEAREST operation, perform a recursive search starting from the root. Whenever a node is reached by the recursive procedure, it is considered visited immediately, and its driver ID must be appended to the Visited list before exploring any child subtree.

During the search, maintain the best driver found so far and its distance to the query point. Then consider the child subtrees in the order SE, SW, NE, NW.

A child subtree should be searched only if the minimum possible Euclidean distance from the query point to that child's subspace is less than or equal to the distance of the current best driver found so far. Otherwise, that subtree must be pruned.

Distance Rule

The distance between a driver located at (x_d, y_d) and a query point (x_q, y_q) must be computed using the standard Euclidean distance formula:

$$\sqrt{(x_f - x_q)^2 + (y_f - y_q)^2}$$

A driver whose distance to the query point is exactly equal to the radius in a REQUEST operation must be considered within range.

Output Rules and Notes

- Drivers must be inserted into the point quadtree in the exact order they appear in *drivers.txt*.
- The PRINT operation must begin with the line Tree:. If the tree is empty, print <Empty> on the next line.
- Each REQUEST and NEAREST operations must produce exactly two lines of output, while ADD and REMOVE produce no output.
- Driver IDs in InRange and Visited outputs must be printed in the exact order determined by the recursive traversal; they must not be sorted.
- A driver may appear in the visited list even if it does not appear in the *InRange* line.
- Driver IDs contain no spaces and must be printed as comma-separated lists with no extra spaces or blank lines. You may also assume that all driver IDs are unique.

Additional Clarifications

- You may assume that all driver IDs in the initial input are unique.
- If a REMOVE operation refers to a driver ID that does not exist in the tree, ignore the operation.
- If a REQUEST operation is performed on an empty tree, print:
  InRange: <None>
  Visited:
- If a NEAREST operation is performed on an empty tree, print:
  Nearest: <None>
  Visited:

Important Note: During grading, the input files *drivers.txt* and *operations.txt* will be provided in the current working directory. Your program is expected to open these files by name and process them accordingly. Do not assume any machine-specific or absolute file path**.**

3. Sample Runs

Below are two sample runs illustrating the expected behavior of the program. Each sample includes the contents of drivers.txt, the contents of operations.txt, and the corresponding program output.

Sample Run 1

drivers.txt :

```
100 100
D1 50 50
D2 70 70
D3 20 80
D4 65 20
```

operations.txt:

```
PRINT
REQUEST 60 60 15
NEAREST 63 18
ADD D5 10 10
REMOVE D1
PRINT
```

Output :

```
Tree:
D1
```

D4
D2
D3
InRange: D1,D2
Visited: D1,D4,D2,D3
Nearest: D4
Visited: D1,D4
Tree:
D4
D5
D2
D3

Sample Run 2

drivers.txt :

50 50
A 25 25
B 40 40

operations.txt:

REQUEST 0 50 5
NEAREST 45 45
PRINT
REMOVE A
PRINT
REMOVE B
PRINT

Output :

InRange: <None>
Visited: A
Nearest: B
Visited: A,B
Tree:
A
B
Tree:
B
Tree:
<Empty>

4. Submission

Your code should be submitted to SUCourse. Please follow the instructions below carefully.

Place your source files in a folder named *XXXX-NameLastname-hw2*, where *XXXX* is your student ID number. Do **not** use any Turkish characters in the folder name.

Your submission folder must contain exactly the following source files:

- source.cpp
- Quadtree.cpp
- Quadtree.h

Do **not** include executable files or unnecessary IDE/build folders such as Debug, Release, .vs, x64, or similar directories.

Compress your submission folder into a single .zip file. For example: *XXXX-NameLastname-hw2.zip* .

Before submitting, make sure that your .zip file opens correctly and reproduces your folder exactly.

Your program will be compiled and tested using the C++17 standard. Make sure that your code is compatible with C++17.

Grading

Your homework will be graded as follows:

- If your program does not use the required **dynamic point quadtree** structure as specified in this assignment, you will receive 0 points, even if your program produces correct output by using another approach.
- We will run **10 tests** on your homework using different input files. Each correct test is worth 10 points, for a total of 100 points.
- Your output must match the required format exactly. Any deviation in formatting may cause a test to fail.

If you have any questions, please contact me at : erfan.rezaei@sabanciuniv.edu